

Last-Minute Revision Sheet – Apple Interview

One-liners only. Skim this 15 mins before you walk in.

Core CS Fundamentals (1-2 lines each)

OOP

- **Encapsulation:** bundle data + methods, hide internals via access modifiers.
- **Abstraction:** expose *what*, hide *how* (interfaces/abstract classes).
- **Inheritance:** child reuses/extends parent behavior – prefer composition when relationship isn't truly "is-a."
- **Polymorphism:** same interface, different behavior – compile-time (overloading) vs runtime (virtual functions/vtable).
- **Virtual function cost:** vtable lookup = indirect call → breaks inlining/branch prediction, avoid on hot paths.

DSA

- **Array vs Linked List:** $O(1)$ random access vs $O(1)$ insert/delete at known position; array cache-friendly, list isn't.
- **Hash map:** $O(1)$ avg via bucket+chaining; degrades on high load factor → triggers $O(n)$ resize.
- **Stack/Queue:** LIFO vs FIFO – stack for recursion/DFS/undo, queue for BFS/scheduling.
- **Tree:** BST $O(\log n)$ avg ops, degrades to $O(n)$ if unbalanced → hence AVL/Red-Black.
- **Graph traversal:** BFS = shortest path (unweighted) + queue; DFS = exploring/cycles + stack/recursion.
- **Sliding window:** $O(n)$ instead of $O(n^2)$ for contiguous subarray problems.
- **Binary search:** $O(\log n)$, needs sorted/monotonic data; boundary search = don't return on first match, keep narrowing.
- **Sorting:** merge sort $O(n \log n)$ stable/predictable; quicksort avg $O(n \log n)$ but $O(n^2)$ worst, in-place.
- **DP:** optimal substructure + overlapping subproblems; top-down (memo) vs bottom-up (tabulation).
- **Big-O reflex:** nested loop over $n = O(n^2)$; hash lookup inside loop = still $O(n)$ not $O(n^2)$.

DBMS

- **ACID:** Atomicity, Consistency, Isolation, Durability – guarantees for a transaction.
- **Isolation levels:** Read Committed (default, PG) → Repeatable Read → Serializable; higher = safer, slower.
- **Indexing:** B-tree for equality/range; speeds reads, slows writes (every index updates on write).
- **Normalization:** reduce redundancy (1NF/2NF/3NF); denormalize deliberately for read-heavy performance.
- **SQL vs NoSQL:** relational/ACID/joins vs flexible schema/horizontal scale; pick by access pattern, not hype.
- **Transactions & locks:** `SELECT FOR UPDATE` to prevent lost updates (e.g., double-debit race).

OS

- **Process vs Thread:** separate memory space vs shared memory within a process; threads cheaper to switch.
- **Deadlock (4 conditions):** mutual exclusion, hold-and-wait, no preemption, circular wait – break any one to prevent.
- **Mutex vs Semaphore:** mutex = binary lock, ownership-bound; semaphore = counter, no ownership, signals across threads.
- **Context switch:** saving/restoring CPU state to switch execution – has real cost, why thread pools > thread-per-request.
- **Virtual memory:** each process gets its own address space via paging; isolates processes, enables swapping.

Networking

- **TCP vs UDP:** reliable/ordered/connection vs fast/connectionless/unreliable – TCP for payments, UDP for streaming.
- **HTTP methods:** GET/PUT idempotent, POST not, DELETE idempotent (same end state).
- **DNS → TCP handshake → TLS → HTTP request:** rough order of what happens when you hit a URL.
- **REST statelessness:** each request carries all context needed; server holds no client session state.

Concurrency

- **Race condition:** unsynchronized concurrent access to shared mutable state → non-deterministic bugs.
- **Mutex:** one thread at a time; **shared_mutex:** many readers OR one writer.
- **Atomic ops:** lock-free, hardware-level (CAS) – cheaper than a mutex for simple counters/flags.
- **Thread pool:** reuse threads instead of spawning per task – avoids OS thread-creation overhead.

Security

- **SQL injection:** fix = parameterized queries, never string concatenation.
 - **XSS:** injected script reads page/cookies → mitigate with HttpOnly cookies + input sanitization.
 - **CSRF:** browser auto-sends cookies cross-site → mitigate with SameSite cookies + CSRF tokens.
 - **Password storage:** bcrypt/Argon2 (slow + salted), never plain SHA-256 (too fast to brute-force).
-

Projects – 30-Second Recaps

HPE – C++ Authorization Service (Feb–Jul 2026)

- Built a **low-latency C++ payment authorization service**: custom thread pool + PostgreSQL connection pool for concurrency, event-driven Kafka microservice for payment routing.
- **Key numbers/claims to restate confidently**: low-latency focus, custom thread pool, Kafka consumer, connection pooling, SQL query optimization.

Invest-Tracker – Portfolio Tracker

- **Stack:** React, Node/Express, Supabase (PostgreSQL).
- **What it does:** live stock/metal prices + news feed, pulling from **3 external APIs** (Finnhub, MetalPriceAPI, Marketaux).
- **Security:** JWT auth + Postgres **Row Level Security** for per-user data isolation.
- **Ops:** deployed on Vercel, fixed CORS misconfig + API edge cases pre-launch.

Quick-Pick – E-Commerce Platform

- **Stack:** React, Node/Express, MongoDB.
- **Scope:** 8 modules – auth, products, cart, orders, reviews, search, admin.
- **Security:** JWT in **HttpOnly cookies** + bcrypt, RBAC middleware (admin/user).
- **Integrations:** Cloudinary (images), PayPal REST API (checkout).
- **Ops:** MongoDB connection caching on Vercel for stability.

Competitive Programming

- Codeforces **Specialist (1455)**, 1000+ problems solved, top 2.1% in a Div 3.
 - Signals: strong algorithmic fundamentals – expect a DSA question that assumes this baseline.
-

Resume Line → Likely Question Mapping

Resume line	What they'll probe
"Custom C++ thread pool"	mutex/condition_variable design, deadlock avoidance, shutdown correctness
"Kafka event-driven microservice"	ordering guarantees, idempotency, at-least-once delivery
"PostgreSQL connection pool"	why pooling, sizing trade-offs, PgBouncer
"Low-latency execution"	be ready to defend with concrete mechanics: no alloc in hot path, cache locality, p99 not avg
"Optimizing SQL queries"	indexing strategy, <code>EXPLAIN ANALYZE</code> , isolation levels
"JWT-based auth ... Row Level Security"	XSS vs CSRF trade-off, RLS bypass pitfalls (service role key)
"JWT ... HTTP-only cookies + bcrypt"	why HttpOnly over localStorage, why bcrypt over SHA-256
"RBAC (admin/user)"	IDOR, broken access control, testing negative-path (403s)
"Finnhub, MetalPriceAPI, Marketaux"	rate limiting 3rd-party APIs, mocking them in tests, caching
"PayPal REST API ... checkout flow"	resilience if provider is down, idempotent order creation
"Resolved CORS misconfigurations"	what CORS actually protects against, preflight, debugging process
"MongoDB connection caching"	schema design (embed vs reference), why caching connections matters on serverless
"Codeforces Specialist"	expect an actual DSA problem – sliding window / binary search / DP / graph
"8 modules" architecture	SOLID principles, how you'd refactor for testability

System Design Quick Recap (1-2 lines each)

- **CAP theorem:** under a network partition, pick Consistency or Availability – can't have both; single-region setups rarely hit this in practice.
- **Consistent hashing:** map nodes+keys onto a ring so adding/removing a node only remaps $\sim 1/N$ of keys, not everything – fixes modulo-hashing's full-reshuffle problem.
- **Load balancing:** round robin (uniform cost) vs least-connections (variable cost) vs consistent-hash routing (needs affinity).
- **Caching strategy:** cache-aside (lazy, DB is source of truth – my default) vs write-through (no staleness, slower writes) vs write-behind (fastest, risks data loss – avoid for financial data).

- **Message queue choice:** Kafka = ordered + replayable + multi-consumer (payment events); RabbitMQ = flexible routing; SQS = simplest, no replay needed.
- **Idempotency:** client-generated idempotency key + server-side check-before-apply – prevents duplicate orders/charges on retry.
- **Rate limiting:** token bucket – allows bursts, enforces steady average; needs shared state (Redis) across multiple instances.
- **Sharding key:** pick based on the dominant query pattern (e.g., `user_id`) so most queries hit one shard; cross-shard aggregation needs a separate analytics path.

Design Patterns / Principles Quick Recap

- **SOLID:** Single responsibility, Open/closed (extend without modifying), Liskov substitution, Interface segregation, Dependency inversion (depend on abstractions).
- **Observer pattern:** publisher doesn't know its subscribers – maps directly to Kafka producer/consumer-group model.
- **Dependency inversion in practice:** code against an interface (`PaymentProvider` , `PriceProvider`) so implementations/mocks swap freely – also what makes testing external APIs possible.

Complexity Cheat Table

Structure	Access	Search	Insert	Delete
Array	$O(1)$	$O(n)$	$O(n)$	$O(n)$
Linked List	$O(n)$	$O(n)$	$O(1)^*$	$O(1)^*$
Hash Map	—	$O(1)$ avg	$O(1)$ avg	$O(1)$ avg
BST (balanced)	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$
Binary Search (sorted array)	—	$O(\log n)$	—	—

*at a known position/node. Sorting: comparison-based lower bound $O(n \log n)$; merge sort stable, quicksort in-place but $O(n^2)$ worst case.

HTTP Status Codes – Quick List

- **200** OK · **201** Created (on successful POST) · **204** No Content (successful DELETE)
- **400** Bad Request (malformed) · **401** Unauthorized (not authenticated) · **403** Forbidden (authenticated, not allowed) · **404** Not Found

- **409** Conflict (e.g., duplicate idempotency key) · **422** Unprocessable Entity (valid syntax, fails business rules)
- **429** Too Many Requests (rate limited) · **500** Internal Server Error · **503** Service Unavailable

Interview-Day Etiquette (coding round)

1. **Clarify before coding** — restate the problem, ask about input size/edge cases (empty input, duplicates, negative numbers).
2. **Think out loud** — narrate your approach and *why*, before writing code, not just while typing.
3. **Start with brute force if stuck**, then optimize — a working $O(n^2)$ beats a broken attempt at $O(n)$.
4. **State time/space complexity unprompted** at the end — don't wait to be asked.
5. **Dry-run your code on a small example** before saying "done" — catches off-by-one errors live.
6. **If you don't know something, say so directly** and reason from first principles rather than guessing confidently.

30-Second Self-Intro (have this ready, don't wing it)

Structure: current role/study → most relevant experience → what you're looking for.

"I'm a CS undergrad at VIT-AP, currently wrapping up a Career Preview Program at HPE where I built a low-latency C++ authorization service for a payment gateway — custom thread pool, Kafka-based event routing, PostgreSQL under it. Outside that I've built two full-stack apps end-to-end — an investment tracker and an e-commerce platform — and I compete on Codeforces, currently rated Specialist. I'm looking for a role where I can keep working at that intersection of systems performance and real product engineering."

Questions to Ask the Interviewer (have 2-3 ready)

- "What does the on-call/incident process look like for this team?"
- "What's the biggest technical challenge the team is working through right now?"
- "How is code review / design review structured here?"

Avoid asking things answerable from the job posting — signals you didn't read it.

60-Second Mental Checklist Before You Walk In

1. Can I say what **low-latency** actually means with 3 concrete techniques? (no hot-path alloc, cache locality, pooled connections)
2. Can I name the **exact anomaly** connection pooling / isolation levels prevent? (lost update / race on balance)

3. Can I explain **why HttpOnly cookies over localStorage** in one breath? (XSS can't read it, trades for CSRF, mitigated by SameSite)
4. Do I have **one real story ready** for "disagreement" and "biggest mistake" (STAR, from HPE or CORS bug)?
5. Do I have a **specific, non-generic reason** for "why Apple / this team"?
6. Am I ready to **think out loud** on a DSA problem — narrate approach before coding, mention complexity unprompted?

Good luck — you've clearly done the depth work. This page is just to keep it retrievable under pressure.